Supplementary Materials of "On the Reliability of Wahoo Formant Measurements: A Response to Boë et al. (2026)"

Frédéric Berthommier

CNRS, Grenoble INP, GIPSA-Lab, Université Grenoble Alpes,
11 Rue des Mathématiques – BP 46, 38402 Saint Martin d'HCedex, Grenoble, France
frederic.berthommier@gipsa-lab.grenoble-inp.fr

October 2026

International Journal of Primatology (2026), accepted 1 October 2026. All material archived in the repository
`github.com/<your-account>/baboon-acoustics`
(interactive demos: `<your-account>.github.io/baboon-acoustics/demos`; Zenodo DOI minted upon first release).

This document describes the software means used to produce every figure of the article and of the present supplement. The *original* scripts, archived verbatim in the folder `supplementary/original_scripts/` of the repository, are described function by function below. They have since been refactored into the documented Python package `baboon_acoustics` (same algorithms, plus tests, examples, user documentation and interactive HTML demos), which is the recommended entry point for reuse; the mapping between the original functions and the package is given in the repository README.

Contents

1 Multi-panel analysis of a baboon wahoo call (`make_figure_multipanel.py`) 1
1.1 Audio loading and resampling . . . 1
1.2 LPC and Levinson–Durbin recursion . . . 3
1.3 Iterative Adaptive Inverse Filtering (IAIF) . . . 3
1.4 Fundamental frequency estimation from the source . . . 3
1.5 Formant extraction from the LPC envelope . . . 3
1.6 Sliding-window IAIF analysis . . . 4
1.7 Vocal tract length estimation . . . 4
1.8 Parselmouth formant extraction . . . 4
1.9 Visualisation utilities . . . 4
1.10 Main figure assembly . . . 5
1.11 Single-pulse extraction . . . 5
1.12 Main execution block . . . 5

2 F0 contour with filtered autocorrelation (`make_figure_ac.py`) 5

3 Analysing the glottal pulse extracted with IAIF (`make_figure_glottal.py`) 7
3.1 Core signal processing functions 7
3.2 Execution and command-line usage 8
3.3 Dependencies 8

4 Re-analysis of the published formant trajectories (`make_figure_1.py`) 8

5 Repository contents 8

1 Multi-panel analysis of a baboon wahoo call (`make_figure_multipanel.py`)

All analyses presented in this section are generated by a single Python script, `make_figure_multipanel.py`. The script depends on NumPy, SciPy, Matplotlib, and Parselmouth (which provides an interface to Praat's algorithms). It reads a WAV file, performs formant estimation via Parselmouth, applies Iterative Adaptive Inverse Filtering (IAIF) to extract the glottal source and vocal-tract filter, and produces the multi-panel figure shown in this supplement (Fig. 1). If no argument is provided, it defaults to `wahoo_4.wav`, which is the last call of the sequence originally recorded in the file `The_Baboon_Bark.wav` and analysed by Boë et al. (2026). The three other calls from the same sequence can be processed instead by passing their respective filenames as a command-line argument. The following subsections describe each major function in the order they appear in the script.

1.1 Audio loading and resampling

```
def load_audio(path)
```

Loads a WAV file using `scipy.io.wavfile.read`, converts stereo to mono by taking the first channel, normalises the signal to the range $[-1, 1]$, and resamples to the target sampling frequency $F_s = 44100$ Hz using SciPy's `resample` function if necessary. It returns the signal as a 1D numpy array and the sampling frequency.

1.2 LPC and Levinson–Durbin recursion

```
def lpc_levinson(x, order)
```

Computes the linear prediction coefficients (LPC) of a given order using the Levinson–Durbin recursion. The input `x` is first mean-centred. The autocorrelation is estimated via FFT for efficiency. The function returns the coefficient vector `a` (with `a[0] = 1`) and the residual variance σ^2. A small regularization is added to avoid numerical instability.

```
def lpc_spectrum_db(a, sigma2, n_fft=4096, fs=FS_TARGET)
```

Given the LPC coefficients and the residual variance, this function computes the frequency response of the all-pole filter $H(f) = \sqrt{\sigma^2} \,/\, A(f)$, where $A(f)$ is the Fourier transform of the coefficient vector. It returns the frequency axis `w` (in Hz) and the magnitude spectrum in dB, computed using `scipy.signal.freqz`.

1.3 Iterative Adaptive Inverse Filtering (IAIF)

```
def iaif_frame(frame, fs, vt_order)
```

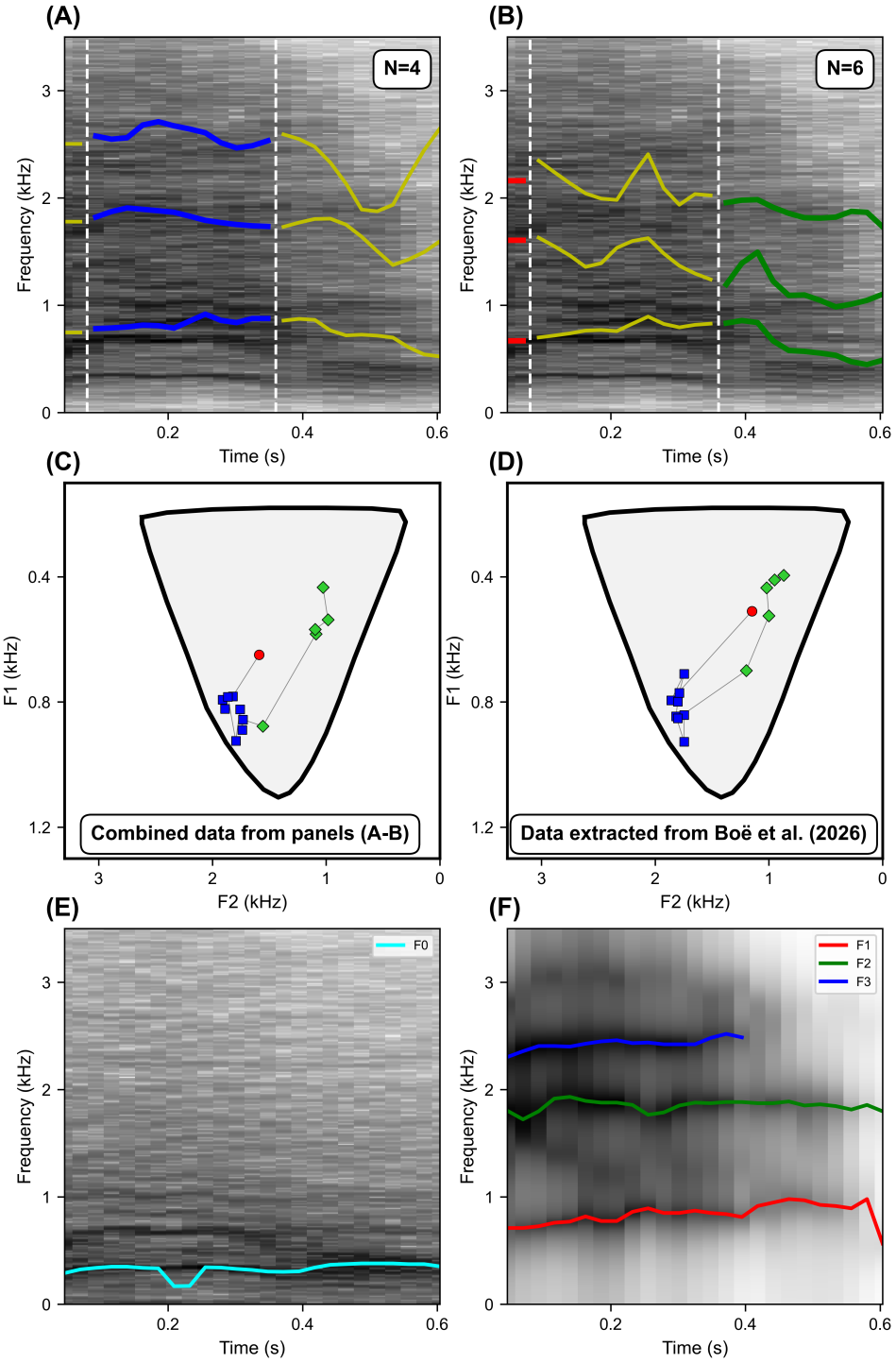


Figure 1: Multi-panel analysis of a baboon wahoo call (exemplar file `wahoo_4.wav`). (A) Wide-band spectrogram of the original signal (Hamming window, length = 4096 samples, hop = 1024 samples, 44.1 kHz sampling rate). Overlaid formant trajectories F1–F3 (yellow before 0.08 s, blue between 0.08 and 0.36 s, yellow again after 0.36 s) were extracted using Praat's Burg algorithm via Parselmouth (*To Formant (burg)*) with parameters: time step = 40 ms, window length = 40 ms, maximum formant = 5500 Hz, pre-emphasis = 50 Hz, number of formants = 4. (B) Same as (A) but with number of formants = 6; formant colours are now red for the first segment, yellow for the middle, green for the last segment. (C) Merged F1–F2 trajectories combining the first point from the $N = 6$ extraction (red circle) and the subsequent points from the $N = 4$ extraction (blue squares and green diamonds). The grey shaded area and black line represent the maximal acoustic space (MAS) contour approximated from Boë et al. (2026). (D) F1–F2 trajectories manually extracted from the spectrogram published in Boë et al. (2026) (red circle, blue squares, green diamonds), superimposed on the same MAS contour. (E) Spectrogram of the glottal source estimate obtained after the second iteration of IAIF (LPC order = 60). The cyan line shows the fundamental frequency (F0) estimated from the source signal by autocorrelation (search range 50–500 Hz, voicing threshold = 0.3). (F) Spectrogram of the vocal-tract filter envelope $|H_{\text{vt}}(f)|$

Performs one frame of the two-iteration IAIF algorithm to separate the glottal source from the vocal-tract filter. The algorithm proceeds as follows:

1. Estimate a first glottal filter H_{g1} of order 1 via LPC, and inverse-filter the frame to obtain s_1.
2. Estimate the vocal-tract filter H_{vt1} of order `vt_order` on s_1.
3. Compute a first source estimate $g_1 = \int(H_{vt1} * frame)$ and remove its mean.
4. Estimate a second glottal filter H_{g2} of order 2 on g_1, inverse-filter the original frame to get s_2.
5. Estimate the final vocal-tract filter H_{vt2} (order `vt_order`) on s_2.
6. Compute the final source signal $g_n = \int(H_{vt2} * frame)$ and remove its mean.

The function returns H_{vt2} (LPC coefficients), σ^2 (residual variance), and the source signal g_n.

1.4 Fundamental frequency estimation from the source

```
def estimate_f0_from_gn(gn, fs, f0_min=F0_MIN, f0_max=F0_MAX, ac_thresh=0.3)
```

Estimates the fundamental frequency F0 from a glottal source segment `gn` by autocorrelation. The signal is mean-centred, then the normalised autocorrelation is computed. The search range for the period is derived from `f0_min` and `f0_max`. The peak of the autocorrelation within this range is found; if its value exceeds `ac_thresh`, the corresponding frequency is returned; otherwise NaN. A simple parabolic interpolation is not performed, but the index is checked to avoid edge cases.

1.5 Formant extraction from the LPC envelope

```
def get_frame_formants(frame, fs, vt_order, fmax, min_prominence, min_dist_hz)
def get_frame_formants_3(frame, fs, vt_order, fmax, min_prominence, min_dist_hz)
```

These functions extract formant frequencies from a given frame by first performing IAIF to obtain the vocal-tract filter H_{vt2}, then computing its magnitude spectrum. Peaks in the spectrum (below `fmax`) are detected using `scipy.signal.find_peaks` with a minimum prominence and a minimum distance between peaks. The four most prominent peaks (sorted by frequency) are returned as F_1, F_2, F_3, F_4. The `get_frame_formants_3` variant returns only three formants and is used in some contexts. If fewer peaks are found, the missing entries are filled with NaN.

1.6 Sliding-window IAIF analysis

```
def iaif_sliding_analysis(x, fs)
def iaif_sliding_analysis_3(x, fs)
```

These functions apply the IAIF analysis to the entire signal using a sliding window (Hamming window of length `FFT_WIN_LEN` and hop size `HOP`). For each frame, they compute:

- the spectrogram of the original signal S_{xx} (dB);
- the spectrogram of the glottal source S_{gn} (dB);
- the magnitude envelope of the vocal-tract filter $|H_{vt2}(f)|$ (dB), interpolated onto a common frequency grid;
- the F0 track (using `estimate_f0_from_gn`);
- the IAIF-based formant tracks (using `get_frame_formants`).

The functions return the time axis, the frequency axis, and the four spectral matrices, as well as the F0 track and the formant matrix. The `_3` variant returns only three formants.

1.7 Vocal tract length estimation

```
def reby_mccomb_vtl_global(F1, F2, F3, F4, c=SPEED_OF_SOUND)
def reby_mccomb_vtl_global_3(F1, F2, F3, c=SPEED_OF_SOUND)
```

Computes a global vocal-tract length (VTL) estimate in cm using the Reby–McComb formula, which fits a linear regression of formant frequencies against odd-order numbers $(1, 3, 5, \ldots)$. The slope b is obtained by $b = \sum f_i \cdot \text{order} \,/\, \sum \text{order}^2$, and then $\text{VTL} = c/(4b)$, where c is the speed of sound in cm/s. Only non-NaN formant values are used. Returns NaN if fewer than two formants are available. The `_3` variant uses only three formants.

1.8 Parselmouth formant extraction

```
def extract_parselmouth_formants_n(audio_path, n_form, target_times,
                                   fs_target=FS_TARGET)
```

Extracts n formants from the audio file using Praat's Burg algorithm (via Parselmouth). The function loads the sound, resamples if necessary, and calls *To Formant (burg)* with the fixed parameters (time step = 40 ms, window length = 40 ms, maximum formant = 5500 Hz, pre-emphasis = 50 Hz). It returns interpolated formant values F_1, F_2, F_3 at the specified `target_times` (time axis from the IAIF analysis), as well as the raw formant tracks at Parselmouth's own time grid. The interpolation handles missing values by using only valid frames.

1.9 Visualisation utilities

```
def plot_f1f2_trajectory(ax, F1, F2, marker_type, marker_color, xlim, ylim)
```

Draws an F2–F1 trajectory on a given Matplotlib axis. It overlays the maximal acoustic space (MAS) contour (grey shaded area and black outline) defined by the hardcoded arrays `MAS_F1_closed` and `MAS_F2_closed`. The trajectory is plotted as a thin grey line, and each point is marked according to the provided `marker_type` and `marker_color`. The axis limits, labels, and tick settings are also configured.

```
def setup_frequency_axis(ax)
def add_panel_label(ax, label)
def fix_initial_nan(arr)
```

`setup_frequency_axis` sets the common style for spectrogram axes: limits (0–3.5 kHz), ticks every 1 kHz on the y-axis and 0.2 s on the x-axis, and labels. `add_panel_label` adds a bold panel label (e.g., "(A)") in the top-left corner of an axis. `fix_initial_nan` replaces a NaN first element of an array by the first non-NaN value, to avoid gaps at the beginning of formant tracks when plotting.

1.10 Main figure assembly

```
def make_figure(x, fs, audio_path, out_path)
```

This is the master function that orchestrates the entire analysis and creates the 3×2 panel figure (Fig. 1). It performs the following steps:

1. Runs `iaif_sliding_analysis` to obtain all IAIF-derived data.

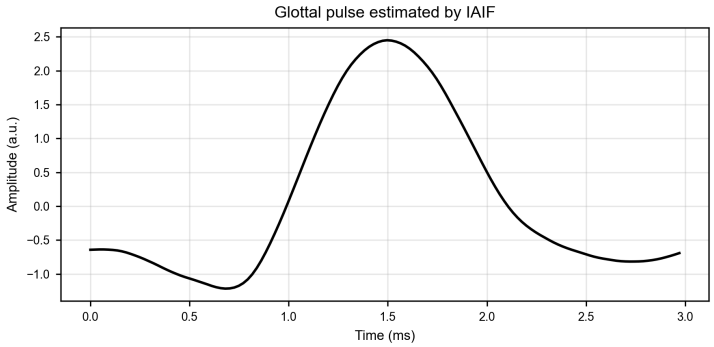


Figure 2: Single glottal pulse estimated by IAIF on the segment 260–340 ms of `wahoo_4.wav` (Hamming window, autocorrelation period, extraction around the largest absolute excursion of the source signal).

2. Computes VTL from the IAIF formants.
3. Extracts Parselmouth formants for $N = 4$ and $N = 6$ using `extract_parselmouth_formants_n`.
4. Builds a 3-row, 2-column grid with `matplotlib.gridspec.GridSpec`.
5. Plots panels (A) through (F) as described in the caption of Fig. 1.
6. Adds panel labels, dashed vertical lines at 0.08 and 0.36 seconds, and a consistent colour map.
7. Saves the figure at 600 dpi.

1.11 Single-pulse extraction

```
def extract_and_save_glottal_pulse(x, fs, png_path)
```

This function extracts a single glottal pulse from a stable portion of the signal (between 0.26 and 0.34 s). It applies IAIF on a Hamming-windowed segment to obtain the source g_n, then estimates the fundamental period via autocorrelation. A single pulse is isolated around the maximum of the absolute source signal, and a time-domain plot is saved as a PNG file (`glottal_pulse.png`, reproduced here as Fig. 2). This function is called separately in the main block.

1.12 Main execution block

The script, when run as a standalone program, reads an audio file either from the command line or uses a default filename (`wahoo_4.wav`). It then loads the audio, calls `extract_and_save_glottal_pulse`, and calls `make_figure` to produce the multi-panel figure shown in this supplement (Fig. 1).

The script is designed to be fully reproducible; all parameters are hardcoded as constants, and the code is provided verbatim in the repository (`supplementary/original_scripts/`). All functions are self-contained and rely only on the imported libraries, ensuring that the entire analysis can be replicated on any system with Python and the required packages installed.

2 F0 contour with filtered autocorrelation (`make_figure_ac.py`)

Figure 3 is produced with the Python program `make_figure_ac.py`. To replicate the behavior of the *filtered autocorrelation* method available in Praat 6.4.65 (but not in the legacy Praat version embedded in Parselmouth 0.4.5), a Gaussian low-pass filter is applied to the time-domain signal prior to pitch extraction. This preprocessing step attenuates high-frequency components above a

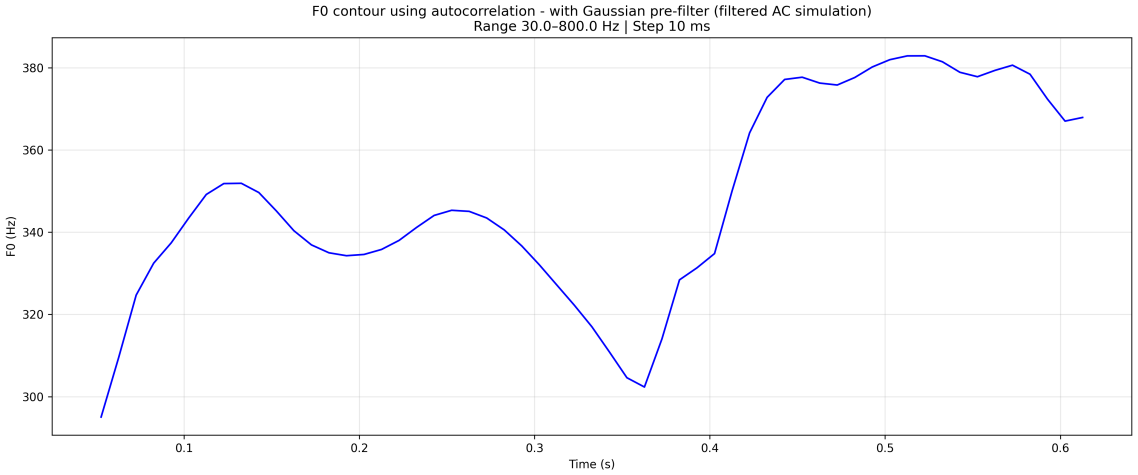


Figure 3: F0 contour obtained after Gaussian low-pass pre-filtering (cutoff 800 Hz, attenuation −30.5 dB at cutoff) followed by Praat's autocorrelation pitch algorithm (*to_pitch_ac*) with floor 30 Hz, ceiling 800 Hz, and time step 0.01 s. The analysis yielded 57 voiced frames, a mean F0 of 348.2 Hz (standard deviation 24.3 Hz). This preprocessing effectively suppresses subharmonics and produces a smooth pitch track comparable to Praat 6.4.65's *To Pitch (filtered ac)…* command.

cutoff frequency f_{cutoff} with a smooth roll-off, effectively removing spectral energy that could otherwise produce subharmonic false peaks in the autocorrelation function.

Subharmonics — often arising from irregular vocal fold vibration or background noise — manifest as additional periodicities at integer fractions of the true fundamental period. By suppressing frequencies above f_{cutoff} (set here to 800 Hz, which is also used as the pitch ceiling in Praat's filtered autocorrelation), the filter preserves the dominant periodicity while reducing the amplitude of spurious subharmonic candidates.

The filter is implemented in the frequency domain using a fast Fourier transform (FFT), applying a Gaussian window $H(f) = \exp(-(f/\sigma)^2/2)$ with $\sigma = f_{\text{cutoff}}/\sqrt{-2\ln(\alpha)}$, where α is the desired attenuation at f_{cutoff} (here $\alpha = 0.03$, corresponding to −30.5 dB). The filtered signal is then transformed back to the time domain via inverse FFT.

This preprocessing step, combined with the standard autocorrelation pitch algorithm (Praat's `to_pitch_ac`) using a pitch floor of 30 Hz, a pitch ceiling of 800 Hz, and a time step of 0.01 s, yields a fundamental frequency contour that closely matches that obtained from Praat 6.4.65's *To Pitch (filtered ac)…* command (Fig. 4), effectively mitigating octave-halving errors and improving robustness in the presence of subharmonics.

3 Analysing the glottal pulse extracted with IAIF (`make_figure_glottal.py`)

This Python script performs a comprehensive analysis of the glottal source waveform extracted from a voiced signal. It uses Iterative Adaptive Inverse Filtering (IAIF) to separate the glottal flow derivative from the vocal-tract filter, then produces a four-panel figure showing the pulse train, phase portrait, spectrogram, and long-term average spectrum (Figs. 5 and 6). The script is designed to run from the command line and requires NumPy, SciPy, Matplotlib, and optionally Parselmouth for resampling.

3.1 Core signal processing functions

`lpc_levinson(x, order)` computes linear prediction coefficients (LPC) using the Levinson–Durbin recursion. Returns the coefficient vector `a` (with `a[0] = 1`) and the residual variance σ^2.

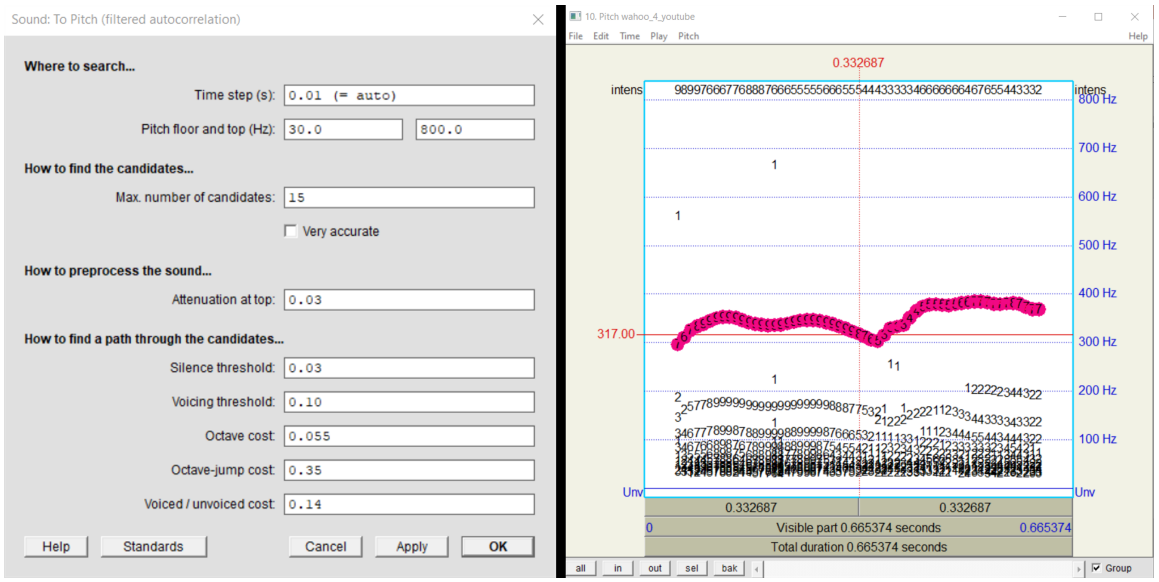


Figure 4: Left: settings of Praat 6.4.65's *To Pitch (filtered autocorrelation)* command. Right: resulting pitch contour of `wahoo_4.wav` displayed in Praat (mean 317 Hz over the visible part). The Python pre-filtering of `make_figure_ac.py` replicates this behaviour.

The autocorrelation is estimated via FFT for efficiency.

`iaif_frame(frame, fs, vt_order)` performs the two-iteration IAIF algorithm on a single frame. It returns the vocal-tract filter coefficients H_{vt2}, the residual variance, and the glottal source signal g_n. The algorithm is described in detail in Section 1 of this supplement.

`estimate_f0_from_gn(gn, fs, f0_min, f0_max, ac_thresh)` estimates the fundamental frequency from the glottal source segment `gn` using autocorrelation. The search range is determined by `f0_min` and `f0_max`; the peak of the normalised autocorrelation must exceed `ac_thresh` (default 0.3) to be accepted. Returns the estimated F0 in Hz, or NaN if no reliable peak is found.

`normalize_amplitude(signal, fs, cutoff, order, eps)` centres and normalises the amplitude of a glottal signal. It removes slow drifts (floating DC) by subtracting a low-pass filtered version, and divides by a low-pass filtered envelope to flatten the overall amplitude. This preprocessing is essential for phase portrait visualisation because it removes amplitude modulation that could obscure the underlying dynamics.

`plot_phase_portrait(signal, fs, title, ax, color, alpha, lw, normalize)` generates a phase portrait in the Hilbert plane. It computes the analytic signal via the Hilbert transform, normalises the real and imaginary parts, and plots the trajectory. If `normalize` is True, the amplitude normalisation is applied first. The function returns the Matplotlib axis object.

`extract_full_glottal_train(x, fs, vt_order, win_len, hop)` reconstructs the full glottal source waveform from the input signal `x` using overlap-add (OLA). The signal is divided into Hann-windowed frames, IAIF is applied to each frame, and the resulting source segments are summed with appropriate window normalisation. Returns the time axis and the reconstructed glottal signal.

`load_audio(path, target_fs)` loads a WAV file, converts to mono, normalises to $[-1, 1]$, and resamples to the target sampling rate (if necessary). Uses Parselmouth if available, otherwise falls back to `scipy.signal.resample_poly`.

3.2 Execution and command-line usage

The script is executed as:

```
python make_figure_glottal.py [audio_file.wav]
```

If no argument is provided, it defaults to `wahoo_4.wav`. The "wa" (100–260 ms) and "hoo" (390–550 ms) analysis segments are selected by the `WA` constant at the top of the file, and the output figure is saved as `glottal_train_analysis_wa.png` or `glottal_train_analysis_hoo.png` accordingly.

3.3 Dependencies

The script requires NumPy, SciPy, and Matplotlib. For automatic resampling, Parselmouth is recommended but not mandatory. All other functions are self-contained. The code is fully reproducible and the parameters can be adjusted at the top of the file to suit different signals or analysis needs.

4 Re-analysis of the published formant trajectories (`make_figure_1.py`)

Figure 1 of the article is produced by `make_figure_1.py`. The script extracts Praat/Burg formant tracks of `wahoo_4.wav` with $N = 4$ and $N = 6$ LPC poles, builds the merged trajectory using the optimal N per segment (first frame from $N = 6$, frames 2–9 from $N = 4$, remaining frames from $N = 6$), reads the reference tracks digitized from Boë et al. (2026, their Fig. 10h), and computes one Reby–McComb VTL estimate per panel. The three panels are drawn with the same comparison-panel style as the article (grey dashed reference with open markers; black estimated tracks with colour-coded initial/medial/final markers) and a shared legend. Running `python make_figure_1.py` reproduces the article figure and prints the four VTL estimates.

5 Repository contents

The complete material is archived in the `baboon-acoustics` repository, of which this supplement is part (github.com/<your-account>/baboon-acoustics; the demos, the present document and the user documentation are also directly readable at <your-account>.github.io/baboon-acoustics):

- `baboon_acoustics/` — refactored, documented Python package implementing the same algorithms (formants Praat/Burg and IAIF, F0, VTL, glottal source, plotting);
- `supplementary/original_scripts/` — the four original scripts described in this document, archived verbatim;
- `supplementary/figures/` — all figures of the article and of the present supplement;
- `data/` — the audio material: the four wahoo calls of the sequence (`wahoo_1.wav` … `wahoo_4.wav`), the full original recording (`The_Baboon_Bark.wav`), and the reference F0 table (`wahoo_4_f0_filtered_ac.csv`);
- `examples/, scripts/, demos/, tests/, docs/` — guided example, figure-reproduction scripts, interactive HTML demos, unit tests and the user documentation of the package.

Reference

Boë, L.-J., Laporte, M., Sawallis, T. R., Badin, P., & Schwartz, J.-L. (2026). Stalking the wild formant: How to reliably measure formants in primate vocalizations. *International Journal of Primatology*, 47(3), 799–841. https://doi.org/10.1007/s10764-026-00551-4

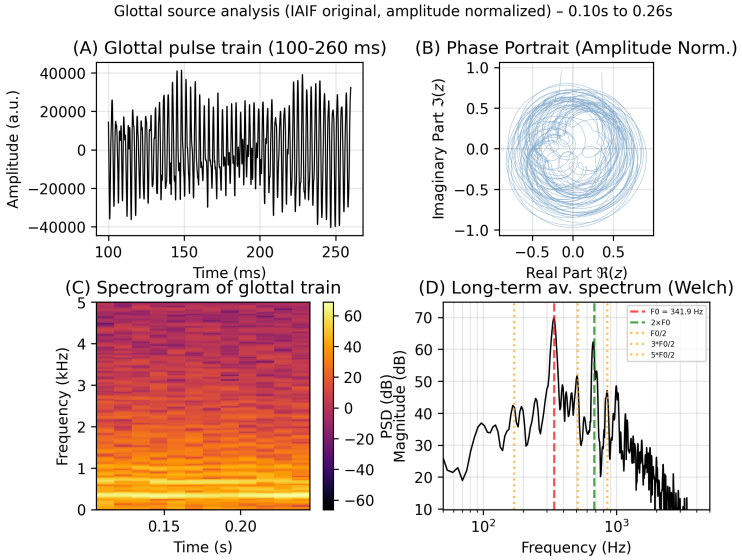


Figure 5: Glottal source analysis (IAIF) — "wa" segment (100–260 ms). (A) Glottal pulse train: the amplitude does not systematically alternate between successive cycles, preventing a definitive conclusion of period doubling. (B) Phase portrait (Hilbert plane): the trajectory forms a thickened annular band rather than a single closed loop. (C) Spectrogram: while a fundamental harmonic and possibly some sub-harmonics are present in the source, the two-frequency dynamics is not visibly apparent. (D) Long-term average spectrum (Welch): in addition to the dominant peak at $F_0 = 341.9\,\text{Hz}$, spectral ridges appear at $F_0/2$, $3F_0/2$, and $5F_0/2$ (orange dotted lines), though their energy is substantially lower than that of the integer harmonics. *Conclusion:* taken together, these features indicate that the glottal source deviates from a purely harmonic and decreasing (e.g., $-12\,\text{dB/octave}$) spectral model, challenging the linear source–filter separation assumption for this segment.

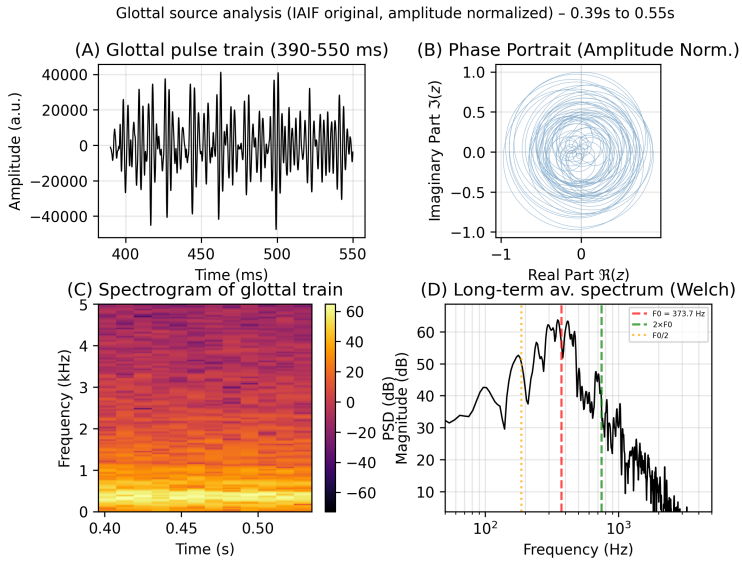


Figure 6: Glottal source analysis (IAIF) — "hoo" segment (390–550 ms). (A) Glottal pulse train: cycle-to-cycle amplitude and timing are strongly irregular with no discernible repeating pattern, indicating a breakdown of any low-period orbital structure present in the earlier segment. (B) Phase portrait (Hilbert plane): the trajectory fills a broad, irregular disk extending beyond unit radius, with no coherent winding structure; this irregular geometry evokes a chaotic-like dynamical regime, although a more precise characterization would require additional analysis. (C) Spectrogram: a broad and thick spectral peak is observed, with a weak harmonic structure, superimposed on a continuous noise floor. (D) Long-term average spectrum (Welch): the fundamental peak is substantially broadened, while a prominent subharmonic peak appears near $F_0/2$ and weak harmonic peaks near $2F_0$; a continuous spectral background is visible between harmonic ridges. *Conclusion:* taken together, these observations indicate a marked departure from a regular periodic source, where the aperiodicity is sufficiently severe to challenge the linear source–filter separation, and therefore any LPC-based formant estimate, for this segment.